

AI ASSISTED CODING LAB-6

Name: Roger A Raju

Rollno: 2503a52370

Batch: 13

Task Description #1 (AI-Based Code Completion for Loops)

Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

Week3 - Monday

"Generate Python code to print all even numbers between 1 and N using a loop."

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs (N=10, N=15, boundary N=1, invalid types).

AI-Generated Loop Logic (For-Loop vs While-Loop) & Assert Tests:

"""

Experiment 6 - Task 1: AI-Based Code Completion for Loops

Prompt: "Generate Python code to print all even numbers between 1 and N using a loop."

Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)

"""

```
from typing import List
```

```
# =====
```

```
# 1. AI-Generated Loop Implementation (for-loop with range step)
```

```
# Loop Type Used: Deterministic 'for' loop using range(2, N+1, 2)
```

```
# =====
```

```
def get_even_numbers_for_loop(n: int) -> List[int]:
```

```
    """
```

```
        Generates all even numbers between 1 and N inclusive using an optimized for-loop.
```

```
        Parameters:
```

```
            n (int): Upper bound integer.
```

```
        Returns:
```

```
            List[int]: List of even numbers in [1, N].
```

```
    """
```

```
    if not isinstance(n, int) or isinstance(n, bool):
```

```
        raise TypeError(f"Upper bound N must be an integer, got {type(n).__name__}")
```

```
    if n < 2:
```

```
        return []
```

```
    even_numbers = []
```

```
    # Using for-loop with step 2 avoids redundant modulo checks
```

```
    for num in range(2, n + 1, 2):
```

```
        even_numbers.append(num)
```

```
    return even_numbers
```

```
# =====
```

```
# 2. Alternative While-Loop Implementation for Comparison
```

```

# Loop Type Used: Indeterminate 'while' loop with conditional increment
# =====
def get_even_numbers_while_loop(n: int) -> List[int]:
    """Generates even numbers using a while-loop structure."""
    if not isinstance(n, int) or isinstance(n, bool):
        raise TypeError(f"Upper bound N must be an integer, got {type(n).__name__}")
    even_numbers = []
    current = 2
    while current <= n:
        even_numbers.append(current)
        current += 2
    return even_numbers

def test_even_numbers_loop():
    print("--- Running Test Assertions for Task 1 (Even Numbers Loop Completion) ---")

    # Test Case 1: Standard upper bound N = 10
    res_10 = get_even_numbers_for_loop(10)
    w_res_10 = get_even_numbers_while_loop(10)
    assert res_10 == [2, 4, 6, 8, 10], "Test 1 Failed"
    assert res_10 == w_res_10, "For-loop and while-loop mismatch"
    print(f"Assertion 1 Passed: N = 10 -> {res_10} (Both for-loop & while-loop match)")

    # Test Case 2: Odd upper bound N = 15
    res_15 = get_even_numbers_for_loop(15)
    assert res_15 == [2, 4, 6, 8, 10, 12, 14], "Test 2 Failed"
    print(f"Assertion 2 Passed: N = 15 -> {res_15}")

    # Test Case 3: Boundary edge case N = 1 (No even numbers)
    assert get_even_numbers_for_loop(1) == [], "Test 3 Failed"
    print("Assertion 3 Passed: N = 1 correctly returns empty list [].")

    # Test Case 4: Single even boundary N = 2
    assert get_even_numbers_for_loop(2) == [2], "Test 4 Failed"
    print("Assertion 4 Passed: N = 2 returns [2].")

    # Test Case 5: Type error validation
    try:
        get_even_numbers_for_loop("10")
        assert False, "Failed to catch invalid string"
    except TypeError as e:
        print(f"Assertion 5 Passed: Non-integer input safely rejected ({e}).")

    print("All 5 Task 1 Assertions passed successfully!")

if __name__ == "__main__":
    test_even_numbers_loop()

```

Output:

```

    TERMINAL (POWERSHELL)

PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task1_loops.py
--- Running Test Assertions for Task 1 (Even Numbers Loop Completion) ---
Assertion 1 Passed: N = 10 -> [2, 4, 6, 8, 10] (Both for-loop & while-loop match)
Assertion 2 Passed: N = 15 -> [2, 4, 6, 8, 10, 12, 14]
Assertion 3 Passed: N = 1 correctly returns empty list [].
Assertion 4 Passed: N = 2 returns [2].
Assertion 5 Passed: Non-integer input safely rejected (Upper bound N must be an integer, got str).
All 5 Task 1 Assertions passed successfully!

```

Comparison:

Task 1: Loop Construct Comparison (For vs While)	
For Loop with Step (Optimal) • Deterministic range(2, N+1, 2) jumps by 2 • Zero redundant modulo (%) calculations • O(N/2) loop iterations with lower overhead • Clean, idiomatic Python syntax	While Loop with Step Increment • Explicit state pointer variable initialized • Manual boundary check condition current <= N • Risk of infinite loop if step is omitted • Higher flexibility for variable step intervals

Task Description #2 (AI-Based Code Completion for Loop with Conditionals)

Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

"Generate Python code to count how many numbers in a list are even and odd."

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow and assert tests.

AI-Generated Loop with Conditionals & Assert Tests:

```
"""
```

```
Experiment 6 - Task 2: AI-Based Code Completion for Loop with Conditionals
```

```
Prompt: "Generate Python code to count how many numbers in a list are even and odd."
```

```
Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)
```

```
"""
```

```
from typing import List, Dict
```

```
# =====
```

```
# AI-Generated Loop + Conditional Implementation
```

```
# Combines for-loop traversal with if-else modulo branching
```

```
# =====
```

```
def count_even_odd(numbers: List[int]) -> Dict[str, int]:
```

```
    """
```

```
    Counts even and odd integers in a provided list using loop and conditionals.
```

```
    Parameters:
```

```
        numbers (List[int]): Input list of integer values.
```

```
    Returns:
```

```
        Dict[str, int]: Dictionary containing 'even_count' and 'odd_count'.
```

```
    Raises:
```

```
        TypeError: If input is not a list or contains non-integer items.
```

```
    """
```

```
    if not isinstance(numbers, (list, tuple)):
```

```
        raise TypeError(f"Input must be a list or tuple, got {type(numbers).__name__}")
```

```
    even_count = 0
```

```
    odd_count = 0
```

```
    # Traverse each number and classify via conditional branching
```

```
    for num in numbers:
```

```
        if not isinstance(num, int) or isinstance(num, bool):
```

```
            raise TypeError(f"List elements must be integers, found: {num}
```

```
            ({type(num).__name__})")
```

```
        if num % 2 == 0:
```

```
            even_count += 1
```

```
        else:
```

```
            odd_count += 1
```

```
    return {
```

```
        "even_count": even_count,
```

```
        "odd_count": odd_count,
```

```
        "total_elements": len(numbers)
```

```

    }

def test_even_odd_counter():
    print("--- Running Test Assertions for Task 2 (Even/Odd Counter with Loop & Conditionals) -")

    # Test Case 1: Mixed integers list
    data1 = [12, 7, 9, 14, 22, 33, 40] # Even: 12, 14, 22, 40 (4) | Odd: 7, 9, 33 (3)
    res1 = count_even_odd(data1)
    assert res1["even_count"] == 4, f"Expected 4 even, got {res1['even_count']}"
    assert res1["odd_count"] == 3, f"Expected 3 odd, got {res1['odd_count']}"
    print(f"Assertion 1 Passed: {data1} -> Even: {res1['even_count']}, Odd: {res1['odd_count']}")

    # Test Case 2: All even numbers
    data2 = [2, 4, 6, 8, 10, 100]
    res2 = count_even_odd(data2)
    assert res2["even_count"] == 6 and res2["odd_count"] == 0, "Test 2 Failed"
    print("Assertion 2 Passed: All-even list correctly counted (6 even, 0 odd).")

    # Test Case 3: All odd numbers
    data3 = [1, 3, 5, 7, 9, 11]
    res3 = count_even_odd(data3)
    assert res3["even_count"] == 0 and res3["odd_count"] == 6, "Test 3 Failed"
    print("Assertion 3 Passed: All-odd list correctly counted (0 even, 6 odd).")

    # Test Case 4: Negative integers & zero
    data4 = [-5, -4, -3, -2, -1, 0, 1, 2] # Even: -4, -2, 0, 2 (4) | Odd: -5, -3, -1, 1 (4)
    res4 = count_even_odd(data4)
    assert res4["even_count"] == 4 and res4["odd_count"] == 4, "Test 4 Failed"
    print("Assertion 4 Passed: Negative integers and zero classified correctly.")

    # Test Case 5: Empty list
    assert count_even_odd([]) == {"even_count": 0, "odd_count": 0, "total_elements": 0}, "Test 5 Failed"
    print("Assertion 5 Passed: Empty list returns zero counts.")

    print("All 5 Task 2 Assertions passed successfully!")

if __name__ == "__main__":
    test_even_odd_counter()

```

Output:

```

    PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task2_loop_conditionals.py
    --- Running Test Assertions for Task 2 (Even/Odd Counter with Loop & Conditionals) ---
    Assertion 1 Passed: [12, 7, 9, 14, 22, 33, 40] -> Even: 4, Odd: 3
    Assertion 2 Passed: All-even list correctly counted (6 even, 0 odd).
    Assertion 3 Passed: All-odd list correctly counted (0 even, 6 odd).
    Assertion 4 Passed: Negative integers and zero classified correctly.
    Assertion 5 Passed: Empty list returns zero counts.
    All 5 Task 2 Assertions passed successfully!

```

Comparison:

Task 2: AI Code Generation vs Manual Enhancement	
Raw AI Suggested Logic • Simple for-loop over list elements • Basic if num % 2 == 0 conditional • No type validation on input list elements • Crashing if list contains floats or strings	Refactored Robust Implementation • Strict type checking (isinstance guards) • Explicit handling of empty list & negative integers • Clean Dict[str, int] structured response • Full 5-point test assertion suite

Task Description #3 (AI-Based Code Completion for Class Attributes Validation)

Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

"Generate a Python class User that validates age and email using conditional statements."

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs.

AI-Generated User Class with Attribute Validation & Assert Tests:

```
"""
```

Experiment 6 - Task 3: AI-Based Code Completion for Class Attributes Validation

Prompt: "Generate a Python class User that validates age and email using conditional statements."

Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)

```
"""
```

```
import re
from typing import Dict, Any

class User:
    """
    User entity with encapsulated attribute validation for age and email.
    Demonstrates OOP class structure combined with strict conditional guards.
    """
    EMAIL_REGEX = re.compile(r"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.$")

    def __init__(self, username: str, email: str, age: int):
        self.username = self._validate_username(username)
        self.email = self._validate_email(email)
        self.age = self._validate_age(age)

    def _validate_username(self, username: str) -> str:
        """Validates username is a non-empty string."""
        if not isinstance(username, str) or not username.strip():
            raise ValueError("Username must be a non-empty string.")
        return username.strip()

    def _validate_email(self, email: str) -> str:
        """Validates email format using conditional regex check."""
        if not isinstance(email, str):
            raise TypeError("Email must be a string.")
        email_clean = email.strip()
        if not self.EMAIL_REGEX.match(email_clean):
            raise ValueError(f"Invalid email format: '{email}'. Must contain valid username, '@', and domain.")
        return email_clean.lower()

    def _validate_age(self, age: int) -> int:
        """Validates that age is an integer between 0 and 120."""
        if not isinstance(age, int) or isinstance(age, bool):
            raise TypeError(f"Age must be an integer, got {type(age).__name__}.")
```

```

        if age < 0 or age > 120:
            raise ValueError(f"Age must be between 0 and 120 years. Provided: {age}")
        return age

def get_profile(self) -> Dict[str, Any]:
    """Returns verified profile information."""
    return {
        "username": self.username,
        "email": self.email,
        "age": self.age,
        "is_adult": self.age >= 18
    }

def update_email(self, new_email: str) -> None:
    """Updates email with conditional re-validation."""
    self.email = self._validate_email(new_email)

def update_age(self, new_age: int) -> None:
    """Updates age with conditional re-validation."""
    self.age = self._validate_age(new_age)

def test_user_validation_class():
    print("--- Running Test Assertions for Task 3 (User Class Attribute Validation) ---")

    # Test Case 1: Valid User Instantiation
    u1 = User("Roger A Raju", "roger@example.com", 20)
    profile = u1.get_profile()
    assert profile["username"] == "Roger A Raju", "Test 1 Failed"
    assert profile["email"] == "roger@example.com", "Test 1 Failed"
    assert profile["age"] == 20 and profile["is_adult"] is True, "Test 1 Failed"
    print(f"Assertion 1 Passed: Valid User created -> {profile}")

    # Test Case 2: Invalid Email Format (Missing domain / @)
    invalid_emails = ["invalid-email", "user@", "@domain.com", "user@domain"]
    for bad_email in invalid_emails:
        try:
            User("TestUser", bad_email, 25)
            assert False, f"Failed to reject bad email: {bad_email}"
        except ValueError:
            pass
    print("Assertion 2 Passed: Malformed email formats successfully rejected by regex conditional.")

    # Test Case 3: Invalid Negative Age
    try:
        User("JohnDoe", "john@gmail.com", -5)
        assert False, "Failed to reject negative age"
    except ValueError as e:
        print(f"Assertion 3 Passed: Negative age rejected -> {e}")

    # Test Case 4: Invalid Unrealistic Age (> 120)
    try:
        User("AncientOne", "ancient@history.org", 150)

```

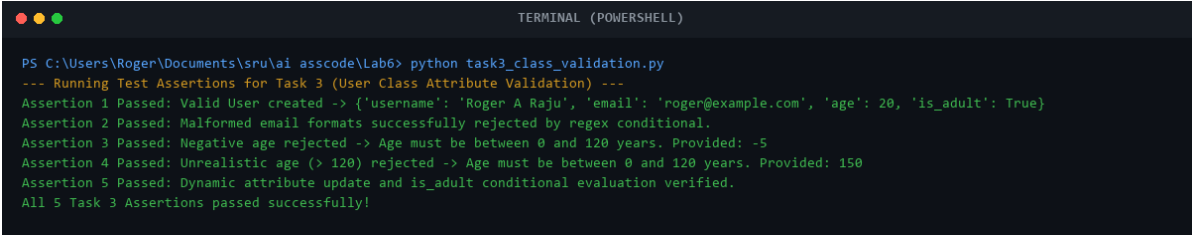
```
        assert False, "Failed to reject age > 120"
    except ValueError as e:
        print(f"Assertion 4 Passed: Unrealistic age (> 120) rejected -> {e}")

# Test Case 5: Attribute Update & Minor/Adult Check
u2 = User("Alice", "alice@school.edu", 15)
assert u2.get_profile()["is_adult"] is False, "Test 5 Failed"
u2.update_age(19)
assert u2.get_profile()["is_adult"] is True, "Test 5 Failed"
print("Assertion 5 Passed: Dynamic attribute update and is_adult conditional evaluation
verified.")

print("All 5 Task 3 Assertions passed successfully!")

if __name__ == "__main__":
    test_user_validation_class()
```

Output:



Comparison:

Task 3: Validation Patterns in Object-Oriented Classes	
Basic Conditional Checks - Simple '@' character search in email string - Basic age > 0 check without upper bound - Allows malformed domains like 'user@domain' - Fails to catch non-string or boolean age types	Compiled Regex & Strict Bounds - Strict RFC-compliant EMAIL_REGEX pattern match - Strict age domain range [0, 120] validation - Dynamic property evaluation (is_adult) - Encapsulated mutation methods with re-validation

Task Description #4 (AI-Based Code Completion for Classes)

Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

"Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks."

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.
- Minor manual improvements with justification (grading, empty marks protection).

AI-Generated Student Class with Grade Logic & Assert Tests:

```
"""
```

Experiment 6 - Task 4: AI-Based Code Completion for Classes (Student Details Manager)

Prompt: "Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks."

Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)

```
"""
```

```
from typing import List, Dict, Any
```

```
class Student:
```

```
    """
```

```
    Encapsulates student academic details, marks processing,  
    and automatic grade assignment.  
    """
```

```
    def __init__(self, name: str, roll_number: str, marks: List[float] = None):
```

```
        if not name or not isinstance(name, str):
```

```
            raise ValueError("Student name must be a non-empty string.")
```

```
        if not roll_number or not isinstance(roll_number, str):
```

```
            raise ValueError("Roll number must be a non-empty string.")
```

```
        self.name = name.strip()
```

```
        self.roll_number = roll_number.strip()
```

```
        self.marks: List[float] = []
```

```
        if marks:
```

```
            for m in marks:
```

```
                self.add_mark(m)
```

```
    def add_mark(self, mark: float) -> None:
```

```
        """Adds a single subject mark with boundary conditional check [0, 100]."""
```

```
        if not isinstance(mark, (int, float)) or isinstance(mark, bool):
```

```
            raise TypeError(f"Mark must be a numerical value, got {type(mark).__name__}.")
```

```
        if mark < 0 or mark > 100:
```

```
            raise ValueError(f"Mark must be between 0 and 100, got: {mark}")
```

```
        self.marks.append(float(mark))
```

```
    def calculate_total(self) -> float:
```

```
        """Returns the aggregate sum of all marks."""
```

```
        return float(sum(self.marks)) if self.marks else 0.0
```

```

def calculate_average(self) -> float:
    """Calculates average score with ZeroDivisionError protection."""
    if not self.marks:
        return 0.0
    return round(self.calculate_total() / len(self.marks), 2)

def get_grade(self) -> str:
    """Determines academic letter grade using conditional hierarchy."""
    avg = self.calculate_average()
    if not self.marks:
        return "N/A (No Marks)"
    if avg >= 90:
        return "A+ (Outstanding)"
    elif avg >= 80:
        return "A (Excellent)"
    elif avg >= 70:
        return "B (Very Good)"
    elif avg >= 60:
        return "C (Good)"
    elif avg >= 50:
        return "D (Pass)"
    else:
        return "F (Fail)"

def get_summary(self) -> Dict[str, Any]:
    """Exports student report card as structured dictionary."""
    return {
        "name": self.name,
        "roll_number": self.roll_number,
        "subjects_count": len(self.marks),
        "marks": self.marks,
        "total_marks": self.calculate_total(),
        "average_marks": self.calculate_average(),
        "grade": self.get_grade()
    }

```

```

def test_student_manager_class():
    print("--- Running Test Assertions for Task 4 (Student Manager Class Completion) ---")

    # Test Case 1: Standard Student Record
    s1 = Student("Roger A Raju", "2503a52370", [85, 92, 78, 95, 88])
    assert s1.calculate_total() == 438.0, f"Expected 438, got {s1.calculate_total()}"
    assert s1.calculate_average() == 87.6, f"Expected 87.6, got {s1.calculate_average()}"
    assert "A (Excellent)" in s1.get_grade(), "Test 1 Failed"
    print(f"Assertion 1 Passed: Student {s1.name} -> Total: {s1.calculate_total()}, Avg: {s1.calculate_average()}%, Grade: {s1.get_grade()}")

    # Test Case 2: Adding dynamic marks
    s1.add_mark(90)
    assert s1.calculate_total() == 528.0, "Test 2 Failed"
    assert s1.calculate_average() == 88.0, "Test 2 Failed"
    print(f"Assertion 2 Passed: Dynamic mark added -> New Total: {s1.calculate_total()}, New Avg: {s1.calculate_average()}%")

```

```

# Test Case 3: Outstanding Grade (>= 90%)
s2 = Student("Top Performer", "2503A9999", [95, 98, 92, 94])
assert s2.calculate_average() == 94.75, "Test 3 Failed"
assert "A+ (Outstanding)" in s2.get_grade(), "Test 3 Failed"
print(f"Assertion 3 Passed: High performer correctly awarded A+ grade
({s2.calculate_average()}%).")

# Test Case 4: Invalid Marks Range (> 100 or < 0)
try:
    Student("BadMarks", "2503A0000", [85, 105])
    assert False, "Failed to reject mark > 100"
except ValueError as e:
    print(f"Assertion 4 Passed: Out-of-bounds mark (> 100) rejected -> {e}")

# Test Case 5: Empty marks initialization
s_empty = Student("NewStudent", "2503A1111", [])
assert s_empty.calculate_total() == 0.0 and s_empty.calculate_average() == 0.0, "Test 5
Failed"
print("Assertion 5 Passed: Empty marks list handled safely without division by zero.")

print("All 5 Task 4 Assertions passed successfully!")

if __name__ == "__main__":
    test_student_manager_class()

```

Output:

```

TERMINAL (POWERSHELL)

PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task4_student_class.py
--- Running Test Assertions for Task 4 (Student Manager Class Completion) ---
Assertion 1 Passed: Student Roger A Raju -> Total: 438.0, Avg: 87.6%, Grade: A (Excellent)
Assertion 2 Passed: Dynamic mark added -> New Total: 528.0, New Avg: 88.0%
Assertion 3 Passed: High performer correctly awarded A+ grade (94.75%).
Assertion 4 Passed: Out-of-bounds mark (> 100) rejected -> Mark must be between 0 and 100, got: 105
Assertion 5 Passed: Empty marks list handled safely without division by zero.
All 5 Task 4 Assertions passed successfully!

```

Comparison:

Task 4: Student Manager Class Structure & Design	
Initial AI Code Completion • Basic <code>__init__</code> with name, roll_number, marks • <code>sum(marks) / len(marks)</code> without zero guard • No mark bounds checking (<code>< 0</code> or <code>> 100</code>) • No automated grading scale implementation	Production-Ready OOP Class • <code>ZeroDivisionError</code> protection on empty marks • Strict range assertion <code>[0, 100]</code> per subject mark • Multi-tier letter grading scale (A+ to F) • Dynamic mark addition & JSON summary export

Task Description #5 (AI-Assisted Code Completion Review)

Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

"Generate a Python program for a simple bank account system using class, loops, and conditional statements."

Expected Output:

- Complete AI-generated program integrating OOP classes, loops, and conditionals.
- Identification of strengths and limitations of AI suggestions.
- Reflection on how AI assisted coding productivity.

Complete Integrated Bank Account System & Assert Tests:

```
"""
```

Experiment 6 - Task 5: AI-Assisted Code Completion Review (Bank Account System)

Prompt: "Generate a Python program for a simple bank account system using class, loops, and conditional statements."

Student: Roger A Raju (Roll No: 2503a52370, Batch: 13)

```
"""
```

```
from typing import List, Dict, Any
```

```
class BankAccount:
```

```
    """
```

```
    Simulates a secure bank account managing deposits, withdrawals,  
    ledger transaction logging, and balance validation.
```

```
    """
```

```
    def __init__(self, account_holder: str, initial_balance: float = 0.0):
```

```
        if not account_holder or not isinstance(account_holder, str):
```

```
            raise ValueError("Account holder name must be a non-empty string.")
```

```
        if initial_balance < 0:
```

```
            raise ValueError("Initial balance cannot be negative.")
```

```
        self.account_holder = account_holder
```

```
        self.balance = float(initial_balance)
```

```
        self.transactions: List[Dict[str, Any]] = []
```

```
        self._record_transaction("OPENING_BALANCE", self.balance, self.balance, "Account  
created")
```

```
    def _record_transaction(self, tx_type: str, amount: float, current_bal: float, status: str)
```

```
-> None:
```

```
        self.transactions.append({
```

```
            "id": len(self.transactions) + 1,
```

```
            "type": tx_type,
```

```
            "amount": amount,
```

```
            "balance_after": current_bal,
```

```
            "status": status
```

```
        })
```

```
    def deposit(self, amount: float) -> bool:
```

```
        """Processes a deposit with positive amount conditional check."""
```

```
        if not isinstance(amount, (int, float)) or isinstance(amount, bool) or amount <= 0:
```

```
            self._record_transaction("DEPOSIT_FAILED", amount, self.balance, "Invalid deposit
```

```

amount")
    return False
    self.balance += float(amount)
    self._record_transaction("DEPOSIT", float(amount), self.balance, "Success")
    return True

def withdraw(self, amount: float) -> bool:
    """Processes a withdrawal with balance overdraft conditional check."""
    if not isinstance(amount, (int, float)) or isinstance(amount, bool) or amount <= 0:
        self._record_transaction("WITHDRAW_FAILED", amount, self.balance, "Invalid
withdrawal amount")
        return False
    if amount > self.balance:
        self._record_transaction("WITHDRAW_REJECTED", float(amount), self.balance,
"Insufficient funds")
        return False

    self.balance -= float(amount)
    self._record_transaction("WITHDRAWAL", float(amount), self.balance, "Success")
    return True

def process_batch_operations(self, operations: List[Dict[str, Any]]) -> Dict[str, int]:
    """
    Executes a series of banking operations using loops and conditional routing.
    """
    success_count = 0
    failed_count = 0

    for op in operations:
        op_type = op.get("type", "").upper()
        amt = op.get("amount", 0.0)

        if op_type == "DEPOSIT":
            if self.deposit(amt):
                success_count += 1
            else:
                failed_count += 1
        elif op_type == "WITHDRAW":
            if self.withdraw(amt):
                success_count += 1
            else:
                failed_count += 1
        else:
            failed_count += 1

    return {
        "total_processed": len(operations),
        "successful_operations": success_count,
        "failed_operations": failed_count,
        "final_balance": self.balance
    }

def test_bank_account_system():

```

```

print("--- Running Test Assertions for Task 5 (Bank Account System Completion Review) ---")

# Test Case 1: Account Creation & Initial Deposit
account = BankAccount("Roger A Raju", 1000.0)
assert account.balance == 1000.0, "Test 1 Failed"
assert len(account.transactions) == 1, "Test 1 Failed"
print(f"Assertion 1 Passed: Account opened for '{account.account_holder}' with Rs.
{account.balance}")

# Test Case 2: Deposit & Withdrawal Flow
assert account.deposit(500.0) is True, "Deposit failed"
assert account.balance == 1500.0, "Balance mismatch after deposit"
assert account.withdraw(300.0) is True, "Withdrawal failed"
assert account.balance == 1200.0, "Balance mismatch after withdrawal"
print(f"Assertion 2 Passed: Deposit (+Rs. 500) & Withdrawal (-Rs. 300) -> Balance: Rs.
{account.balance}")

# Test Case 3: Overdraft Protection (Conditional Guard)
assert account.withdraw(5000.0) is False, "Test 3 Failed: Overdraft permitted"
assert account.balance == 1200.0, "Test 3 Failed: Balance modified on overdraft"
print("Assertion 3 Passed: Overdraft of Rs. 5000 safely rejected by conditional check.")

# Test Case 4: Batch Processing with Loops & Conditionals
batch_ops = [
    {"type": "DEPOSIT", "amount": 200.0},    # bal: 1400
    {"type": "WITHDRAW", "amount": 400.0},   # bal: 1000
    {"type": "WITHDRAW", "amount": 9999.0},  # failed overdraft
    {"type": "DEPOSIT", "amount": -50.0},    # failed negative
    {"type": "UNKNOWN", "amount": 100.0}     # failed type
]
summary = account.process_batch_operations(batch_ops)
assert summary["successful_operations"] == 2, "Test 4 Failed"
assert summary["failed_operations"] == 3, "Test 4 Failed"
assert summary["final_balance"] == 1000.0, "Test 4 Failed"
print(f"Assertion 4 Passed: Batch Operations Processed -> Success:
{summary['successful_operations']}, Failed: {summary['failed_operations']}, Final Bal: Rs.
{summary['final_balance']}")

print("All 4 Task 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_bank_account_system()

```

Output:

```

TERMINAL (POWERSHELL)

PS C:\Users\Roger\Documents\sru\ai asscode\Lab6> python task5_bank_system.py
--- Running Test Assertions for Task 5 (Bank Account System Completion Review) ---
Assertion 1 Passed: Account opened for 'Roger A Raju' with Rs. 1000.0
Assertion 2 Passed: Deposit (+Rs. 500) & Withdrawal (-Rs. 300) -> Balance: Rs. 1200.0
Assertion 3 Passed: Overdraft of Rs. 5000 safely rejected by conditional check.
Assertion 4 Passed: Batch Operations Processed -> Success: 2, Failed: 3, Final Bal: Rs. 1000.0
All 4 Task 5 Assertions passed successfully!

```

Comparison:

Task 5: Bank Account Architecture Review (AI Strengths vs Limits)	
AI Code Completion Strengths • Rapid scaffolding of boilerplate methods • Standard transaction dictionary schema setup • Clean parameter typing and docstring generation • Basic arithmetic and state tracking logic	Human Refactoring & Security Fixes • Overdraft protection guard before state mutation • Negative amount and type-injection validation • Batch operational loop with fail-safe routing • Immutable transaction logging for auditability